

6b - Programmazione MPI - applicazioni

Parallelizzazione di CPI con MPI

Nella directory `mpi/cpi/` è disponibile `mpi_cpi.c` che parallelizza CPI con MPI.

Tecnica di parallelizzazione: scomposizione del dominio 1D in sottodomini

L'intervallo di integrazione tra 0 e 1 viene suddiviso tra i task: ogni task individua il proprio sottodominio in funzione del proprio rank. In questo caso `Lfirst` e `Llast` sono gli indici locali degli intervalli da calcolare:

```
Lfirst=(n/mpiTasks)*mpiRank+1;  
Llast=(n/mpiTasks)*mpiRank+(n/mpiTasks);
```

Nella directory `mpi/cpi/` trovate esempi di script `slurm` e `python` per l'analisi della scaling con 4 task distribuiti su un singolo nodo (`mpi_cpi-1n.slurm`) e su 2 nodi (`mpi_cpi-2n.slurm`)

Sistemare ed eseguire gli script sul PC personale o sul cluster HPC.

- sul PC personale: `sh mpi_cpi-1n.slurm`
- sul cluster: `sbatch mpi_cpi-1n.slurm`

Gli script `mpi_cpi-2n` mettono a confronto le performance tra le diverse distribuzioni dei 4 task su 1 nodo (1x4) o su 2 nodi (2x2).

Caricare su Elly gli output, i csv e i file png.

Parallelizzazione di Heat con MPI

Nella directory `mpi/heat/` si trova il sorgente `mpi_heat.c` e gli script `slurm` e `python` per lo studio della scalabilità con 4 core in 1 nodo e con 4 core distribuiti su 2 nodi.

Analizzare nel sorgente la tecnica di decomposizione, il ruolo delle primitive `MPI_Scatter()`, `MPI_Gather()`, `MPI_Sendrecv()` e gli overhead della parallelizzazione; sul cluster o sul PC personale eseguire gli script, generare i plot e caricare su Elly output, csv e png.

Le seguenti estensioni di MPI Heat possono essere implementate o comunque valutate

Checkpoint

Nel sorgente `mpi_heat.c` le funzioni `save_heatmap()` e `load_heatmap()` consentono di salvare e caricare lo stato della heatmap verso il file `heatmap.txt`. Opzionalmente creare `mpi_heat_checkpoint.c` che include le funzioni nel ciclo di esecuzione e testare il funzionamento.

Caricate su Elly l'eventuale file `mpi_heat_checkpoint.c` funzionante.

Overlap comunicazione/computazione

I punti interni della superficie che un task deve gestire possono essere elaborati anche se i dati dei bordi non sono ancora stati ricevuti dai task adiacenti. Il sorgente `mpi_heat.c` include funzioni `Jacobi_Iterator_CPU_internal()` e `Jacobi_Iterator_CPU_external()` che gestiscono rispettivamente l'aggiornamento dei punti interni (`internal`) e dei punti adiacenti al bordo (`external`).

Realizzare o studiare la fattibilità del file `mpi_heat_overlap.c` in cui lo scambio delle righe e l'aggiornamento dei punti può avvenire in sovrapposizione temporale, inserendo correttamente le seguenti istruzioni in sostituzione delle esistenti:

```
MPI_Isend( &(h_T_old[NX*(NY-2)]), NX, MPI_FLOAT, next_rank,  
tag, MPI_COMM_WORLD, &requestS1);  
MPI_Irecv( &(h_T_old[NX*0]), NX, MPI_FLOAT, prev_rank,  
tag, MPI_COMM_WORLD, &requestR1);  
MPI_Isend( &(h_T_old[NX*1]), NX, MPI_FLOAT, prev_rank,  
tag, MPI_COMM_WORLD, &requestS2);  
MPI_Irecv( &(h_T_old[NX*(NY-1)]), NX, MPI_FLOAT, next_rank,  
tag, MPI_COMM_WORLD, &requestR2);
```

```
MPI_Wait( &requestS1, &status);  
MPI_Wait( &requestS2, &status);  
MPI_Wait( &requestR1, &status);  
MPI_Wait( &requestR2, &status);
```

```
Jacobi_Iterator_CPU_external(h_T_old, h_T_new, NX, NY);
```

```
Jacobi_Iterator_CPU_internal(h_T_old, h_T_new, NX, NY);
```

Caricate su Elly l'eventuale file `mpi_heat_overlap.c` funzionante.

Programmazione ibrida mpi+openMP

Nelle moderne architetture hardware ibride (multicore + multinodo + GPU) per ottimizzare l'utilizzo delle risorse disponibili occorre l'utilizzo contemporaneo di diversi modelli di programmazione. Se disponiamo di diversi nodi di calcolo possiamo utilizzare i thread di openMP all'interno di ogni nodo, mentre MPI viene utilizzato per la comunicazione tra i nodi. Se disponiamo di diverse GPU possiamo utilizzare CUDA all'interno di ogni GPU e MPI per la comunicazione tra le GPU. In generale questa programmazione ibrida viene denominata MPI+X.

Vedi <https://www.hpcwire.com/2014/07/16/compiler-mpix/>

La directory mpi/heat/MPI+openMP/ include i file per questa attività.

Nel sorgente mpi+omp_heat.c ogni task MPI utilizza openMP per parallelizzare il ciclo for di Jacobi.

Lo script mpi+omp_heat.slurm richiede 8 cores (2x4):

- richiede due nodi (--nodes=2)
- su ogni nodo viene eseguito un task MPI (--ntasks-per-node=1)
- per ogni task MPI vengono richieste 4 CPU (--cpus-per-task=4) che verranno utilizzate da 4 thread openMP.

La distribuzione tra task e thread può essere forzata manualmente (vedi ultime righe dello script).

Opzionale: confrontare le performance di scaling di heat nelle diverse tecniche di parallelizzazione (openMP, mpi e mpi+omp) Vedi ad esempio mpi+omp_heat_scaling.slurm e mpi+omp_heat_scaling.py.

Caricare su Elly gli output, i csv e i file png.

Parallelizzazione di Factorize con MPI

Nella directory mpi/factorize si trova il sorgente mpi_factorize.c che gestisce Check-pointing, ovvero la possibilità (con opzione -s) di riprendere l'analisi da un blocco specifico.

Il sorgente mpi_factorize_irecv.c aggiunge la gestione della comunicazione tra il task che trova il numero primo e gli altri task.

Verificare nel sorgente il flusso della comunicazione:

- Ogni task attiva inizialmente la ricezione non bloccante di un messaggio con `MPI_Irecv()`, quindi prima di analizzare un blocco verifica con `MPI_Test()` se ha ricevuto un messaggio.
- Il task che trova il numero primo invia un messaggio a tutti con `MPI_Send()`.

Sistemare ed eseguire lo script `mpi_factorize.slurm`, sul cluster HPC o sul pc personale e caricare sul Elly gli output.

- sul cluster `sbatch mpi_factorize.slurm`
- sul PC personale `sh      mpi_factorize.slurm`